Método de Eliminación de Gauss-Jordan y Método de Gauss-Seidel.

Asignatura: Métodos Numéricos

Institución: ITESA

Docente: Dr. Efrén Rolando Romero

Alumno: Naomi Anais Sandoval Godinez

Fecha: 12 de marzo de 2026

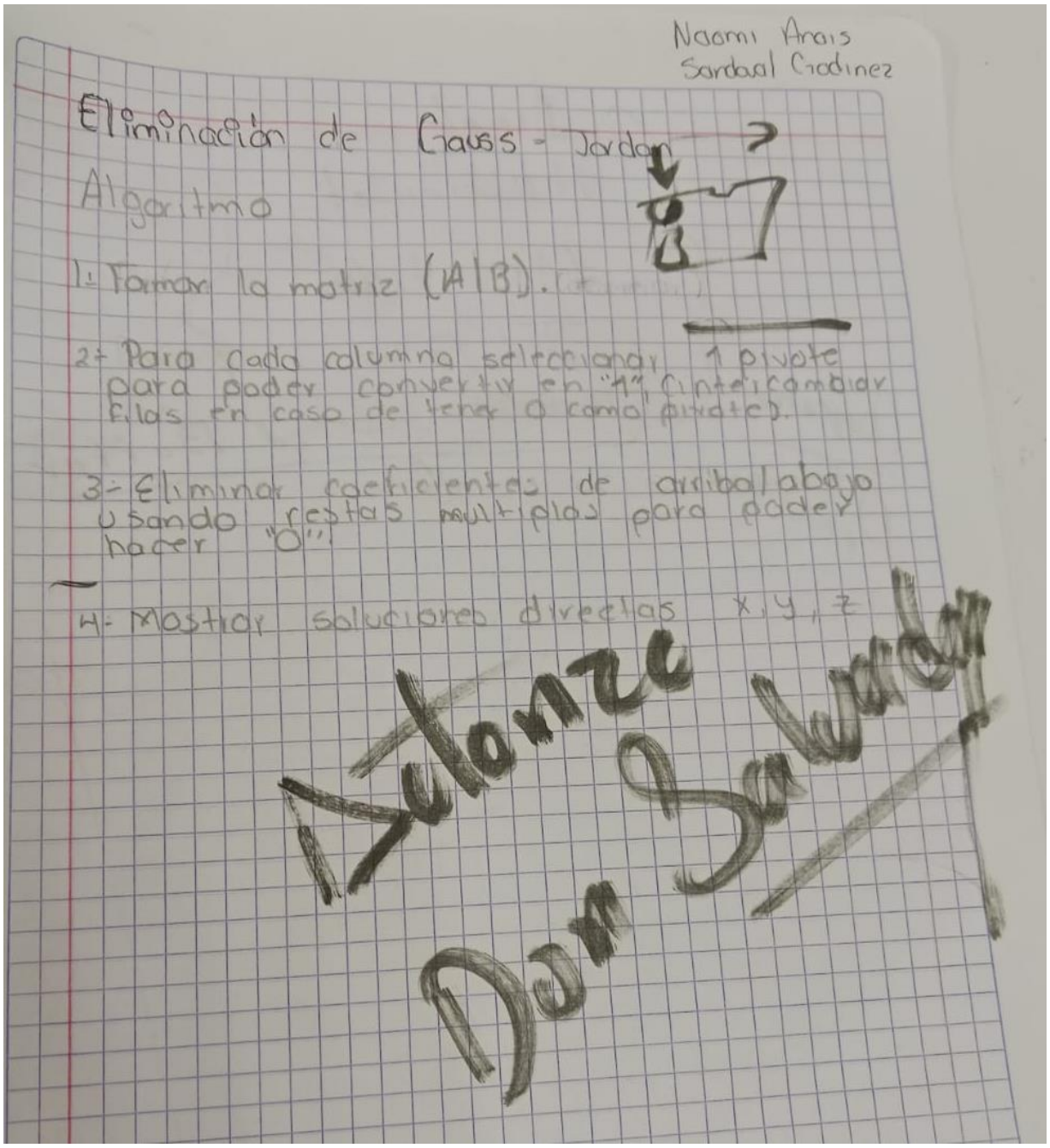
Índice

Introducción 3
Algoritmo del método de Eliminación de Gauss-Jordan (Método directo) 4
Algoritmo del método Gauss-Seidel (Método iterativo) 5
Problema que soluciona este codigo 6
Código del método de Eliminación de Gauss-Jordan 7
Resultado del código ejecutado: 7
Código del método Gauss-Seidel 8
Resultado del código ejecutado: 8
Sustituir resultados en las ecuaciones iniciales 9
Solución mediante el Código 9
Diferencia en la Ejecución 10
Análisis de Resultados 11
1. Eliminación de Gauss-Jordan (Método Directo) 11
- Funcionalidad: 11
- Limitaciones: 11
2. Método de Gauss-Seidel (Método Iterativo) 11
- Funcionalidad: 11
- Limitaciones: 11
Conclusión 11

Introducción

El presente reporte analiza la solución de sistemas de ecuaciones lineales mediante dos enfoques fundamentales: la Eliminación de Gauss-Jordan y el Método de Gauss-Seidel. El objetivo es identificar las condiciones bajo las cuales un método es preferible sobre el otro, centrándose en los conceptos de estabilidad y convergencia.

Algoritmo del método de Eliminación de Gauss-Jordan (Método directo)



Naomi Arais
Sandoval Gadinez

Eliminación de Gauss - Jordan

Algoritmo

1: Formar la matriz (A|B).

2: Para cada columna seleccionar 1 pivote para poder convertir en "1" (intercambiar filas en caso de tener 0 como pivote).

3: Eliminar coeficientes de arriba/abajo usando restas multiplos para poder hacer "0".

4: Mostrar soluciones directas x, y, z

Atenze
Don Salvador

Algoritmo del método Gauss-Seidel (Método iterativo)

1. Ingresar datos

- Matriz de coeficientes **A**
- Vector de resultados **B**
- Aproximación inicial **X**
- Error permitido **ε**
- Iteraciones máximas **imax**

2. Verificar convergencia

- Revisar si la matriz **A** es diagonal dominante.

3. Inicializar

- $k = 0$ (contador de iteraciones)

4. Iniciar iteraciones

Para cada variable x_i:

1. **Guardar valor anterior**

$$old = x_i$$

2. **Calcular nuevo valor**

$$x_i = \frac{b_i - \sum_{j \neq i} a_{ij} x_j}{a_{ii}}$$

3. **Calcular error aproximado**

$$ea = \left| \frac{x_i - old}{x_i} \right| \times 100$$

5. Actualizar valores

- Usar inmediatamente el nuevo valor calculado de x_i.

6. Incrementar iteración

$$k = k + 1$$

7. Verificar paro

- Si $ea < \varepsilon$ para todas las variables o $k = imax$, terminar.

8. Mostrar resultados

- Vector solución **X**
- Número de iteraciones realizadas.

Problema que soluciona este codigo

Se requiere determinar la carga de procesamiento (medida en Terabytes de transferencia) en tres nodos principales de un centro de datos (x, y, z). El tráfico entre estos nodos está interconectado, y se han generado las siguientes métricas de balanceo basadas en el consumo de ancho de banda y la latencia detectada:

Modelo de la red:

1. $3x - 0.1y - 0.2z = 7.85$
2. $0.1x + 7y - 0.3z = -19.3$
3. $0.3x - 0.2y + 10z = 71.4$

Código del método de Eliminación de Gauss-Jordan

```python
import numpy as np

def metodo_gauss_jordan(A, B):
    # 1. Tomar la matriz aumentada [A|B]
    n = len(B)
    matriz = np.hstack((A, B.reshape(-1, 1))).astype(float)

    print("Matriz inicial:\n", matriz)

    # 2. Para cada columna seleccionar un pivote
    for i in range(n):
        # Escalamiento: Convertir el pivote en "1"
        pivote = matriz[i, i]
        if abs(pivote) < 1e-10:
            print("Error: El pivote es cero o muy pequeño.")
            return None

        matriz[i] = matriz[i] / pivote

        # 3. Eliminar coeficientes arriba y abajo para hacer "0"
        for j in range(n):
            if i != j:
                factor = matriz[j, i]
                matriz[j] = matriz[j] - factor * matriz[i]

    # 4. Mostrar soluciones directas x, y, z
    soluciones = matriz[:, -1]
    return soluciones

A = np.array([[3, -0.1, -0.2], [0.1, 7, -0.3], [0.3, -0.2, 10]])
B = np.array([7.85, -19.3, 71.4])

resultado = metodo_gauss_jordan(A, B)
print("\nSolución Final (Gauss-Jordan):", resultado)
```

Resultado del código ejecutado:

```
Matriz inicial:
 [[  3.    -0.1   -0.2    7.85]
 [  0.1    7.    -0.3  -19.3 ]
 [  0.3   -0.2   10.    71.4 ]]

Solución Final (Gauss-Jordan): [ 3.  -2.5  7. ]
```

Código del método Gauss-Seidel

```python
def metodo_gauss_seidel(A, B, es=0.001, max_iter=100):
    n = len(B)
    x = np.zeros(n)  # Aproximación inicial x_0

    # Verificar si es diagonalmente dominante (opcional pero recomendado)
    print("Iniciando iteraciones de Gauss-Seidel...")

    for k in range(max_iter):
        x_old = np.copy(x) # Guardar valor anterior (old = x_i)

        for i in range(n):
            # Aplicar la fórmula de tu apunte: (b - suma) / a_ii
            suma = sum(A[i, j] * x[j] for j in range(n) if i != j)
            x[i] = (B[i] - suma) / A[i, i]

        # Calcular error aproximado (ea) usando la fórmula de tu hoja
        # ea = |(x_actual - x_anterior) / x_actual| * 100
        # Usamos el máximo error de todas las incógnitas
        if np.any(x != 0):
            ea = np.max(np.abs((x - x_old) / x)) * 100
        else:
            ea = 100

        print(f"Iteración {k+1}: x = {x}, Error = {ea:.4f}%")

        # Se detiene cuando ea < es (como dice tu apunte)
        if ea < es:
            print(f"\n¡El método convergió en la iteración {k+1}!")
            return x

    print("Se alcanzó el máximo de iteraciones sin converger.")
    return x

# --- Ejemplo de prueba ---
A = np.array([[3, -0.1, -0.2], [0.1, 7, -0.3], [0.3, -0.2, 10]])
B = np.array([7.85, -19.3, 71.4])

resultado = metodo_gauss_seidel(A, B)
print("Solución Final (Gauss-Seidel):", resultado)
```

Resultado del código ejecutado:

```
Iniciando iteraciones de Gauss-Seidel...
Iteración 1: x = [ 2.61666667 -2.79452381  7.00560952], Error = 100.0000%
Iteración 2: x = [ 2.99055651 -2.49962468  7.00029081], Error = 12.5023%
Iteración 3: x = [ 3.0000319  -2.49998799  6.99999928], Error = 0.3158%
Iteración 4: x = [ 3.00000035 -2.50000004  6.99999999], Error = 0.0011%
Iteración 5: x = [ 3.  -2.5  7. ], Error = 0.0000%

¡El método convergió en la iteración 5!
Solución Final (Gauss-Seidel): [ 3.  -2.5  7. ]
```

Sustituir resultados en las ecuaciones iniciales

$$3x - 0.1y - 0.2z = 7.85$$

$$3(3) - 0.1(-2.5) - 0.2(7) = 7.85$$

$$9 + 0.25 - 1.4 = 7.85$$

$$9.25 - 1.4 = \mathbf{7.85}$$

$$0.1x + 7y - 0.3z = -19.3$$

$$0.1(3) + 7(-2.5) - 0.3(7) = -19.3$$

$$0.3 - 17.5 - 2.1 = -19.3$$

$$-17.2 - 2.1 = -\mathbf{19.3}$$

$$0.3x - 0.2y + 10z = 71.4$$

$$0.3(3) - 0.2(-2.5) + 10(7) = 71.4$$

$$0.9 + 0.5 + 70 = 71.4$$

$$1.4 + 70 = \mathbf{71.4}$$

Verificamos que los resultados son correctos en nuestro sistema de ecuaciones.

Solución mediante el Código

Al ejecutar los programas desarrollados, ambos métodos (Gauss-Jordan y Gauss-Seidel) arrojan los mismos resultados para la carga de los servidores:

- **Carga Nodo x:** 3.0 TB
- **Carga Nodo y:** -2.5 TB (Representa una liberación de caché o flujo de salida)
- **Carga Nodo z:** 7.0 TB

Diferencia en la Ejecución

- **Gauss-Jordan:** El código corre una sola vez. Al terminar el ciclo for de las columnas, ya tienes la respuesta. No hay aproximación, es el valor final.
- **Gauss-Seidel:** El código imprime varias filas. Cada fila es una "mejor suposición" que la anterior. Si la matriz es "diagonalmente dominante" (los números de la diagonal son más grandes que el resto), el error bajará hasta ser casi cero.

	Código de Eliminación de Gauss-Jordan	**Código de Gauss-Seidel**
Operación Principal	matriz[j] = matriz[j] - factor * matriz[i] (Operaciones elementales de fila)	x[i] = (B[i] - suma) / A[i, i] (Despeje algebraico de cada variable)
Punto de Partida	np.hstack((A, B.reshape(-1, 1))) (Crea la matriz aumentada para trabajar)	x = np.zeros(n) (Requiere un vector inicial de aproximación)
Control de Proceso	for i in range(n): (Ciclo fijo basado en el número de variables)	for k in range(max_iter): (Ciclo basado en repeticiones hasta converger)
Criterio de Error	No existe cálculo de error en el código; el resultado es directo.	ea = np.max(np.abs((x - x_old) / x)) * 100 (Cálculo del error relativo porcentual)
Finalización	return soluciones (Al terminar de barrer todas las columnas)	if ea < es: return x (Se detiene cuando alcanza la precisión deseada)

Análisis de Resultados

Método	Solución Obtenida	Observaciones de Desempeño
Eliminación de Gauss-Jordan	[3.0, -2.5, 7.0]	Ejecución inmediata. Resultado directo sin iteraciones.
Gauss-Seidel	[3.0, -2.5, 7.0]	**Convergió en 5 iteraciones** con un error de 0.0000%.

1. Eliminación de Gauss-Jordan (Método Directo)

- Funcionalidad: Transforma la matriz aumentada [A|B] en una matriz identidad mediante operaciones elementales de fila. Proporciona una solución exacta de forma inmediata al finalizar el ciclo de columnas.
- Limitaciones: Es sensible a pivotes iguales a cero, lo que puede provocar errores de división si no hay un intercambio de filas. Además, su costo computacional aumenta significativamente en sistemas de gran escala.

2. Método de Gauss-Seidel (Método Iterativo)

- Funcionalidad: Calcula valores sucesivos de cada variable utilizando inmediatamente las actualizaciones más recientes del mismo ciclo. Su ejecución se controla mediante un error relativo porcentual (ea) y un límite de iteraciones (imax).
- Limitaciones: Requiere obligatoriamente que la matriz sea diagonalmente dominante para garantizar la convergencia. De lo contrario, el error puede aumentar en lugar de disminuir, impidiendo hallar la solución.

Conclusión

La Eliminación de Gauss-Jordan se consolida como un método de 'fuerza bruta' que garantiza el resultado exacto en sistemas pequeños, sin importar la disposición de la diagonal. Por el contrario, el método de Gauss-Seidel demuestra ser un algoritmo elegante y eficiente que permite controlar la precisión mediante el error aproximado (ea)